

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant est capable de :

- Lire et écrire des fichiers texte en Python
- Manipuler le format CSV (lecture, écriture, analyse)
- Utiliser le module csv de Python
- Traiter des données réelles (notes d'élèves, résultats)
- Générer des rapports automatisés
- Créer des outils pédagogiques réutilisables (fiches de révision, journal de bord)

Plan du TP

Phase	Durée	Activité
Accueil et révision	5 min	Rappel sur les fichiers
Exercice 1	15 min	Lecture et écriture de fichiers texte
Exercice 2	20 min	Création d'un fichier CSV de notes
Exercice 3	20 min	Analyse de données CSV
Exercice 4	20 min	Générateur de fiches de révision
Exercice 5	10 min	Journal des séances

Exercice 1 — Lecture et écriture de fichiers texte (15 min)

Énoncé : Créer un script `journal.py` qui :

1. Demande à l'utilisateur d'entrer 3 phrases (son journal du jour)
2. Sauvegarde chaque phrase dans un fichier `journal.txt` (une phrase par ligne)
3. Lit le fichier et affiche son contenu numéroté

Exemple attendu :

Phrase 1 : Aujourd'hui j'ai appris les fichiers en Python.
Phrase 2 : C'est utile pour sauvegarder des données.
Phrase 3 : Je vais créer un projet de quiz.

```
=== Mon journal ===  
1. Aujourd'hui j'ai appris les fichiers en Python.  
2. C'est utile pour sauvegarder des données.  
3. Je vais créer un projet de quiz.
```

Corrigé :

```
with open("journal.txt", "w", encoding="utf-8") as f:
    for i in range(3):
        phrase = input(f"Phrase {i+1} : ")
        f.write(phrase + "\n")

print("\n=== Mon journal ===")
with open("journal.txt", "r", encoding="utf-8") as f:
    for i, ligne in enumerate(f, 1):
        print(f"{i}. {ligne.strip()}")
```

Exercice 2 — Création d'un fichier CSV de notes (20 min)

Énoncé : Créer un script `creer_notes.py` qui :

1. Demande le nombre d'élèves
2. Pour chaque élève, demande nom, note1, note2, note3
3. Sauvegarde dans `notes.csv` avec en-tête
4. Affiche un récapitulatif des données enregistrées

```
flowchart LR
    A[Saisie utilisateur] --> B[notes.csv]
    B --> C[Lecture par analyse_notes.py]
    C --> D[Calcul moyennes]
    D --> E[resultats.csv]
    style B fill:#f9f,stroke:#333,stroke-width:2px
    style E fill:#9cf,stroke:#333,stroke-width:2px
```

Corrigé :

```
import csv

nb = int(input("Nombre d'élèves : "))
eleves = []

for i in range(nb):
    print(f"\n--- Élève {i+1} ---")
    nom = input("Nom : ")
    n1 = float(input("Note 1 : "))
    n2 = float(input("Note 2 : "))
    n3 = float(input("Note 3 : "))
    eleves.append([nom, n1, n2, n3])

with open("notes.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Nom", "Note1", "Note2", "Note3"])
    writer.writerows(eleves)

print(f"\n✓ {nb} élèves enregistrés dans notes.csv")
```

Exercice 3 — Analyse de données CSV (20 min)

Énoncé : Créer un script `analyse_notes.py` qui lit `notes.csv` et calcule :

1. La moyenne de chaque élève
2. La moyenne de la classe
3. Le nombre d'élèves admis (moyenne ≥ 10)

4. Exporte les résultats dans resultats.csv

Exemple attendu :

```
=== Analyse des notes ===
Ali → Moyenne : 14.00 – ✓ Admis
Sara → Moyenne : 16.67 – ✓ Admis
Mohamed → Moyenne : 9.00 – x Ajourné
Leila → Moyenne : 16.00 – ✓ Admis
Ahmed → Moyenne : 9.33 – x Ajourné

Moyenne de la classe : 13.00
Élèves admis : 3/5
Résultats exportés dans resultats.csv
```

Corrigé :

```
import csv

try:
    with open("notes.csv", "r", encoding="utf-8") as f:
        pass
except FileNotFoundError:
    with open("notes.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Nom", "Note1", "Note2", "Note3"])
        writer.writerow(["Ali", 14, 12, 16])
        writer.writerow(["Sara", 18, 15, 17])
        writer.writerow(["Mohamed", 8, 10, 9])
        writer.writerow(["Leila", 16, 14, 18])
        writer.writerow(["Ahmed", 10, 11, 7])

    eleves = []
    with open("notes.csv", "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for ligne in reader:
            notes = [float(ligne["Note1"]), float(ligne["Note2"]), float(ligne["Note3"])]
            moyenne = sum(notes) / len(notes)
            eleves.append({"nom": ligne["Nom"], "notes": notes, "moyenne": moyenne})

    with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Nom", "Note1", "Note2", "Note3", "Moyenne", "Admis"])
        for e in eleves:
            admis = "Oui" if e["moyenne"] >= 10 else "Non"
            writer.writerow([e["nom"], e["notes"][0], e["notes"][1],
                             e["notes"][2], f"{e['moyenne']:.2f}", admis])

    moyennes = [e["moyenne"] for e in eleves]
    moyenne_classe = sum(moyennes) / len(moyennes)
    admis = sum(1 for m in moyennes if m >= 10)

    print("=== Analyse des notes ===")
    for e in eleves:
        etat = "✓ Admis" if e["moyenne"] >= 10 else "x Ajourné"
        print(f"{e['nom']:<10} → Moyenne : {e['moyenne']:.2f} – {etat}")

    print(f"\nMoyenne de la classe : {moyenne_classe:.2f}")
    print(f"Élèves admis : {admis}/{len(eleves)}")
    print("Résultats exportés dans resultats.csv")
```

Exercice 4 — Générateur de fiches de révision (20 min)

Énoncé : Créer un script `fiches_revision.py` qui permet de créer et d'utiliser des fiches de révision. Le programme :

1. Charge les fiches depuis un CSV (concept, définition)
2. Lance un quiz : affiche le concept, l'utilisateur donne la définition
3. Marque comme correcte ou non
4. Affiche le score final

```

flowchart TD
    A[Début] --> B[Créer / charger fiches CSV]
    B --> C[Mélanger les fiches]
    C --> D[Afficher concept]
    D --> E[Utilisateur saisit définition]
    E --> F[Afficher définition attendue]
    F --> G[Auto-évaluation : Correct ?]
    G --> H{Plus de fiches ?}
    H -->|Oui| D
    H -->|Non| I[Afficher score final]
    I --> J[Fin]
    style B fill:#f9f,stroke:#333,stroke-width:2px

```

Corrigé :

```

import csv
import random

def creer_fiches(nom_fichier):
    """Crée un fichier CSV de fiches de révision."""
    concepts = [
        ("Variable", "Boîte qui stocke une valeur"),
        ("Fonction", "Bloc de code réutilisable qui effectue une tâche"),
        ("Boucle", "Structure qui répète une instruction"),
        ("Condition", "Structure qui exécute selon qu'une condition est vraie"),
        ("Tableau", "Collection d'éléments accessibles par indice"),
        ("Pile", "Structure LIFO – dernier entré, premier sorti"),
        ("File", "Structure FIFO – premier entré, premier sorti"),
        ("Algorithme", "Suite finie d'instructions pour résoudre un problème")
    ]
    with open(nom_fichier, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Concept", "Definition"])
        writer.writerows(concepts)
    print(f"{len(concepts)} fiches créées dans {nom_fichier}")

def charger_fiches(nom_fichier):
    """Charge les fiches depuis un fichier CSV."""
    fiches = []
    with open(nom_fichier, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for ligne in reader:
            fiches.append((ligne["Concept"], ligne["Definition"]))
    return fiches

def lancer_quiz(fiches):
    """Lance un quiz de révision."""
    random.shuffle(fiches)
    score = 0
    print("\n=== Quiz de révision ===\n")
    for concept, definition in fiches:
        print(f"Concept : {concept}")
        reponse = input("Votre définition : ")
        print(f"Définition attendue : {definition}")
        correct = input("Correct ? (o/n) : ").lower()

```

```

if correct == "o":
    score += 1
print()
print(f"Score final : {score}/{len(fiches)}")

fichier = "fiches_revision.csv"
creer_fiches(fichier)
fiches = charger_fiches(fichier)
lancer_quiz(fiches)

```

Exercice 5 — Journal des séances (10 min)

Énoncé : Créer un script `journal_bord.py` qui enregistre un journal de travail horodaté. Le programme :

1. Demande le nom de l'étudiant
2. Demande les matières étudiées et le temps passé
3. Sauvegarde dans un CSV avec horodatage automatique
4. Affiche un résumé de la session

Exemple attendu :

```

Nom de l'étudiant : Sara
Matière étudiée (ou 'fin') : Maths
Temps passé sur 'Maths' (min) : 45
Matière étudiée (ou 'fin') : Python
Temps passé sur 'Python' (min) : 60
Matière étudiée (ou 'fin') : fin

=== Résumé de la session du 2026-06-25 ===
Étudiant : Sara
Nombre de matières : 2
• Maths : 45 minutes
• Python : 60 minutes
Temps d'étude total : 105 minutes (1.8 heures)
Données sauvegardées dans journal_bord.csv

```

Corrigé :

```

import csv
from datetime import date

nom = input("Nom de l'étudiant : ")
date_jour = date.today()

matieres = []
while True:
    matiere = input("Matière étudiée (ou 'fin') : ")
    if matiere == "fin":
        break
    temps = int(input(f" Temps passé sur '{matiere}' (min) : "))
    matieres.append((matiere, temps))

with open("journal_bord.csv", "a", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    if f.tell() == 0:
        writer.writerow(["Date", "Nom", "Matiere", "Temps_min"])
    for matiere, temps in matieres:
        writer.writerow([date_jour, nom, matiere, temps])

```

```

temps_total = sum(t for _, t in matieres)
print(f"\n=== Résumé de la session du {date_jour} ===")
print(f"Étudiant : {nom}")
print(f"Nombre de matières : {len(matieres)}")
for m, t in matieres:
    print(f" • {m} : {t} minutes")
print(f"Temps d'étude total : {temps_total} minutes ({temps_total/60:.1f} heures)")
print("Données sauvegardées dans journal_bord.csv")

```

QCM d'auto-évaluation

- Quelle fonction Python permet d'ouvrir un fichier ?
 - `open()` ✓
 - `file()`
 - `read()`
- Que signifie CSV ?
 - Comma-Separated Values ✓
 - Computer System Validation
 - Common Source Variables
- Quel mode d'ouverture permet d'écrire à la fin d'un fichier existant ?
 - "w"
 - "a" ✓
 - "r"
- Quelle méthode lit toutes les lignes d'un fichier CSV sous forme de dictionnaires ?
 - `csv.reader()`
 - `csv.DictReader()` ✓
 - `csv.writer()`
- Pourquoi utilise-t-on `newline=""` avec `csv.writer()` ?
 - Pour éviter les lignes vides entre les rangées ✓
 - Pour supprimer les caractères spéciaux
 - Ce n'est pas nécessaire

Grille d'évaluation TP 07

Critère	Poids	Non acquis (0)	Partiel (0.5)
Lecture/écriture fichiers texte	20 %	Ne maîtrise pas <code>open()</code>	Utilisation correcte
Module CSV (lecture)	20 %	Ne sait pas lire un CSV	Lecture avec <code>DictReader</code>
Module CSV (écriture)	20 %	Ne sait pas écrire un CSV	Écriture avec <code>writer</code>
Traitement de données	20 %	Aucun traitement	Traitement des données
Export de résultats	20 %	Pas d'export	Export partiel

Déroulement séance — 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 — Journal texte	15 min	20 min
Exercice 2 — Création CSV notes	20 min	40 min
Exercice 3 — Analyse CSV	20 min	60 min
Exercice 4 — Fiches révision	20 min	80 min
Exercice 5 — Journal de bord	10 min	90 min
Total	90 min	

Note pédagogique ENS

Ce TP est le plus concret des huit séances : il manipule des données réelles qui parlent aux futurs enseignants.

- Exercice 2 et 3 (notes CSV) : Ces exercices simulent le travail quotidien d'un enseignant : saisir des notes, calculer des moyennes, exporter des résultats. Les étudiants réalisent qu'un script Python peut remplacer un tableur pour des tâches répétitives. En classe de collège/lycée, on peut remplacer les notes par des résultats sportifs ou des données météo.
- Exercice 4 (fiches de révision) : C'est l'exercice le plus directement réutilisable. Les futurs enseignants peuvent créer des fiches pour n'importe quelle matière (histoire, langues, sciences). Le format CSV permet de mutualiser les fiches entre enseignants. Prolongement possible : ajouter un système de pondération (fiches mal maîtrisées reviennent plus souvent).
- Exercice 5 (journal de bord) : Idéal pour enseigner les bonnes pratiques d'étude. Les élèves peuvent suivre leur temps de travail sur la durée. L'horodatage automatique évite les erreurs de saisie.
- Ouverture interdisciplinaire : Proposer aux étudiants de créer un fichier CSV de vocabulaire français-anglais, puis un script de quiz pour leurs futurs élèves en langues.

Le traitement de fichiers est une compétence transversale utile dans toutes les matières scientifiques.